

Processamento de Linguagens - Construção de um Compilador para Fortran 77 Standard

Trabalho realizado por:

- Juliana Silva A105572
- Sofia Couto A106925
- Soraia Pereira A106806

Índice

1. [Introdução](#)
2. [Estrutura do Projeto](#)
3. [Análise Léxica](#)
 - [3.1. Palavras Reservadas e Tokens](#)
4. [Análise Sintática e Gramática](#)
 - [4.1. Declarações e Tipos](#)
 - [4.2. Hierarquia de Expressões](#)
 - [4.3. Construção da Árvore de Sintaxe Abstrata \(AST\)](#)
5. [Análise Semântica](#)
6. [Tradução para Código Máquina](#)
7. [Testes Realizados](#)
8. [Dificuldades Encontradas e Limitações Atuais](#)
9. [Otimizações implementadas](#)
 - [9.1. Constant Folding](#)
 - [9.2. Eliminação de Código Morto](#)
 - [9.3. Remoção de Ciclos Mortos](#)
 - [9.4. Eliminação de Dupla Negação](#)
10. [Instruções de Execução](#)
11. [Conclusão](#)

1. Introdução

Este projeto foi desenvolvido no âmbito da unidade curricular de Processamento de Linguagens e tem como objetivo a construção de um compilador para a linguagem Fortran 77 Standard. O compilador implementado recebe como entrada um ficheiro com código Fortran 77 e realiza as várias fases de compilação: análise léxica, análise sintática, análise semântica e tradução para código máquina da EWVM.

No projeto desenvolvido, foi utilizada a biblioteca PLY em Python, nomeadamente os módulos `ply.lex` e `ply.yacc`, para implementar o analisador léxico e o analisador sintático.

2. Estrutura do Projeto

A implementação foi organizada em vários ficheiros, cada um associado a uma fase ou responsabilidade específica do compilador:

```
.
├── lexer.py           # Analisador léxico
├── parser.py          # Analisador sintático e construção da AST
├── symbol_table.py    # Tabela de símbolos e análise semântica
├── translator.py      # Tradução da AST para código EWVM
├── errors.py          # Classes de erro utilizadas no projeto
├── tests/             # Programas Fortran 77 de teste
└── output/            # Código máquina gerado para a EWVM
```

O ficheiro `errors.py` define exceções próprias, como `ParseError`, `SemanticError` e `SemanticWarning`, permitindo separar erros sintáticos, semânticos e avisos de análise.

A separação em módulos torna o projeto mais organizado: o `lexer.py` identifica os tokens, o `parser.py` valida a gramática e constrói a AST, o `symbol_table.py` gere identificadores, tipos, escopos e validações semânticas, e o `translator.py` transforma a AST em instruções da EWVM.

3. Análise Léxica

A análise léxica foi implementada com a biblioteca `PLY` que gera um analisador léxico a partir de expressões regulares. O lexer percorre o código-fonte em Fortran77 e produz sequências de tokens.

É importante mencionar que, antes de iniciar a tokenização, é feita uma validação da indentação de código através da função `check_indentation()`. Esta função é responsável por verificar que o código segue as regras do Fortran77:

- Comentários: Identifica linhas que comecem em `C`, `c` ou `*` e ignora-as;
- Labels: entre as colunas 1 e 5 só são aceites inteiros, que correspondem ao token `LABEL`. Se a função detetar que nesse espaço estão caracteres que não sejam dígitos lança um `LexError`.

3.1. Palavras Reservadas e Tokens

Os tokens são todas as unidades léxicas que o lexer pode produzir. Neste projeto inclui:

- Labels numéricas: `LABEL`
- Literais inteiros e reais: `INT`, `NREAL`
- Booleanos: `BOOL`
- Strings: `STRING`
- Identificadores de variáveis: `VAR`
- Operadores aritméticos, relacionais e lógicos: `OPADDSUB`, `OPDIV`, `POWER`, `CONCAT`, `LT`, `GT`, `LE`, `GE`, `EQ`, `NE`, `AND`, `OR`, `NOT`
- Operador de atribuição: `EQUALS`
- Linhas de continuação: `CONTINUATION`
- Literais de caracteres únicos: `(`, `)`, `,`, `*`, `'`

Espaços e tabulações são ignorados.

As palavras reservadas são um subconjunto dos tokens que, como o nome indica, têm um significado fixo na linguagem que não pode ser usado para outros fins, como nomear variáveis. Estas palavras incluem:

- Instruções de controlo de fluxo: **IF**, **THEN**, **ELSE**, **ENDIF**, **DO**, **GOTO**, **CONTINUE**, **RETURN**, **STOP**
- Declarações de unidades de programa: **PROGRAM**, **END**, **FUNCTION**, **SUBROUTINE**, **CALL**
- Declaração de Variáveis ou Constantes: **INTEGER**, **REAL**, **LOGICAL**, **CHARACTER**, **PARAMETER**
- Instruções de entrada/saída: **PRINT**, **READ**
- Funções Intrínsecas: **MOD**

Quando o lexer lê um identificador, independentemente de estar minúsculas ou maiúsculas, verifica se pertence ao conjunto de palavras reservadas e, caso detete que pertence, atualiza o seu tipo para o token correspondente. Para além disso se identifica que existe um caracter na coluna 6, é detetado como continuação da linha anterior.

Se algum token não for reconhecido, é lançada um **LexError** com a linha em que ocorreu.

4. Análise Sintática e Gramática

A análise sintática é a segunda fase do processo de compilação, responsável por verificar se a sequência de tokens produzida pelo analisador léxico está organizada de acordo com as regras gramaticais da linguagem. Neste projeto, o analisador sintático foi implementado em Python com recurso à biblioteca PLY, que gera um parser LALR(1) a partir das regras de produção definidas. É importante referir também que é o parser quem chama as funções de verificação de semântica da `symbol_table`.

4.1. Declarações e Tipos

O ponto de arranque é o **ProgramUnitList**, que representa o ficheiro Fortran composto por uma ou mais unidades de programa, onde cada uma pode ser um **PROGRAM**, uma **FUNCTION** ou uma **SUBROUTINE**. Cada unidade começa com um cabeçalho que cria um novo âmbito na tabela de símbolos, contém uma lista de instruções e termina com **END**.

```
ProgramUnitList : ProgramUnit | ProgramUnitList ProgramUnit
ProgramUnit    : Program | FunctionDeclaration | SubroutineDeclaration
Program        : ProgramHeader StatementList END
ProgramHeader  : PROGRAM VAR
```

São suportados os tipos **INTEGER**, **REAL**, **LOGICAL** e **CHARACTER** para declaração de variáveis, que podem ser simples ou arrays unidimensionais com declaração de tamanho. Para além disso, também é suportado declaração de constantes através da palavra reservada **PARAMETER**, sendo o seu valor obrigatoriamente uma expressão verificável em tempo de compilação.

4.2. Hierarquia de Expressões

As expressões seguem uma hierarquia, representada na tabela abaixo, onde categorias com nível mais alto têm precedência sob as de níveis mais baixos:

Nível	Categoria	Operadores
1	Lógico	.OR.

Nível	Categoria	Operadores
2	Lógico	.AND.
3	Negação lógica	.NOT.
4	Relacional	.LT. .GT. .LE. .GE. .EQ. .NE.
5	Aditivo	+, -
6	Multiplicativo	*, /, MOD(...)
7	Potência	**
8	Concatenação	//
9	Elemento	variáveis, literais, chamadas, outras expressões

Esta hierarquia é a responsável por garantir, entre outras, que somas não são efetuadas antes de multiplicações na mesma expressão e está expressa na gramática através de não-terminais encadeados. Assim não há a necessidade de declarações explícitas de precedência:

```

Expression      : LogicalTerm | Expression OR LogicalTerm
LogicalTerm     : LogicalFactor | LogicalTerm AND LogicalFactor
LogicalFactor   : NOT LogicalFactor | NonLogicalExpression
NonLogicalExpression: AdditiveExpression | AdditiveExpression RelationalOp
AdditiveExpression
AdditiveExpression : Term | AdditiveExpression OPADDSUB Term
Term              : PowerExpression | Term OPDIV PowerExpression
                  | Term "*" PowerExpression | MOD "(" Term ", "
PowerExpression  ")"
PowerExpression  : ConcatenationExpression | PowerExpression POWER
ConcatenationExpression
ConcatenationExpression : ExpressionElement | ConcatenationExpression
CONCAT ExpressionElement
ExpressionElement : VAR | IndexOrCall | INT | NREAL | BOOL | STRING | "("
Expression ")"

```

É importante notar que o **IndexOrCall** é ambíguo a nível sintático já que tanto o acesso a arrays como a chamada de funções partilham a sintaxe **VAR(args)**. Assim, a sua distinção é feita na fase de análise semântica, onde a tabela de símbolos determina se o identificador corresponde a um array ou a uma função.

4.3. Construção da Árvore de Sintaxe Abstrata (AST)

À medida que as regras são validadas, o parser constrói nós em formato de tuplos, gerando uma árvore que representa o programa de forma estruturada. Para uma melhor explicação, foi gerado o exemplo abaixo:

O código

```

IF (A .LT. B) THEN
    X = 10

```

```
ELSE
    X = 20
ENDIF
```

gera a seguinte AST:

```
(
  'IF',
  ('LT', 'A', 'B'),
  [('ASSIGN', 'X', 10)],
  [('ASSIGN', 'X', 20)]
)
```

5. Análise Semântica

A análise semântica foi implementada principalmente através da classe `SymbolTable`, que guarda duas estruturas de dados principais: uma lista de escopos e uma tabela de símbolos. Os símbolos guardados na tabela de símbolos incluem variáveis de tipos simples, arrays, parâmetros (as constantes definidas por `PARAMETER`), labels, parâmetros formais de funções e subrotinas e os valores de retorno de funções. No nosso projeto, cada símbolo é registado com informação relevante para realizar a validação semântica, nomeadamente o seu índice, tipo, estado de inicialização, tamanho (para o caso dos arrays) e ainda flags que indicam se o símbolo corresponde a um `PARAMETER`, argumento formal, valor de retorno de uma função ou label.

```
self.__table[name] = {
    'index': idx,
    'type': var_type,
    'initialized': False,
    'is_array': is_array,
    'size': size,
    'is_parameter': is_parameter,
    'is_formal_param': is_formal_param,
    'is_return_value': is_return_value,
    'is_label': is_label,
    'value': value
}
```

A lista de escopos está representada por duas estruturas: uma "stack" com os nomes dos escopos e um dicionário que mapeia cada nome de escopo para um dicionário contendo informação sobre o nome do escopo, a tabela de símbolos associada, o nome do escopo anterior, o tipo do escopo (programa, função ou subrotina), o tipo de retorno caso seja uma função, uma flag que indica se o valor de retorno da função já foi atribuído em algum ponto do código e um campo para guardar o endereço de retorno (usado na fase de tradução). O escopo global corresponde ao escopo usado para o programa principal. Quando se entra numa regra de produção de uma unidade de programa, mais especificamente nas regras de produção das headers para as funções e subrotinas, é chamada a função `push_scope` que guarda a tabela de símbolos do escopo

anterior na lista de escopos e cria uma tabela nova vazia para o novo escopo. Quando se sai da regra de produção, é chamada a função `pop_scope` que guarda a tabela de símbolos no dicionário do escopo atual na lista de escopos e atualiza para o escopo anterior. Embora a implementação de escopos tenha sido feita sobre a forma de uma stack, esta estrutura acabou por não ser tão útil como esperado, uma vez que o Fortran 77 não tem suporte a blocos aninhados.

A estrutura permite centralizar várias verificações importantes. Por exemplo, quando uma variável é declarada, a Symbol Table verifica se já existe uma declaração com o mesmo nome no escopo atual, evitando declarações duplicadas. Quando é feita uma atribuição, verifica-se se a variável foi previamente declarada, se não corresponde a uma constante definida por `PARAMETER`, e se o tipo do valor atribuído é compatível com o tipo da variável. No caso dos arrays, também é verificado se o acesso é feito com um índice válido e do tipo inteiro.

Para as expressões, recorreu-se ao uso da função auxiliar `get_expr_type` que, dado um nodo da AST, usa recursividade para determinar o tipo de uma expressão, usando tipagem automática para valores constantes e recorrendo à tabela de símbolos para variáveis. A própria função levanta erros semânticos quando deteta incompatibilidades de tipos em operações ou uso de variáveis não declaradas ou inicializadas.

As labels definidas por `GOTO` e `DO` são registadas para verificação posterior. No final da análise, o compilador verifica se todos os saltos apontam para labels existentes. Também é verificado se a variável de controlo do ciclo `DO` é inteira e se os limites do ciclo são numéricos.

Todas as funções e subrotinas têm de redeclarar os seus parâmetros formais dentro do seu escopo, para que seja possível obter os seus tipos. No fim da análise, é verificado se todas as funções e subrotinas chamadas foram declaradas, e se os parâmetros usados nas chamadas são compatíveis em número e tipo com os parâmetros formais declarados. No caso das funções, é verificado se em algum ponto do código é atribuído um valor de retorno à função antes do comando `RETURN`.

6. Tradução para Código Máquina

A tradução para código máquina da EWVM é feita no ficheiro `translator.py` e corresponde à fase final do compilador. Depois da análise léxica, sintática e semântica, é criado um objeto `Translator` com a `symbol_table`. Este objeto percorre a AST gerada pelo parser e transforma cada nó numa lista de instruções em código máquina, que no fim é gravada no diretório `output`.

Dentro do `translator.py`, a classe `Translator` percorre cada unidade do programa chamando funções específicas como `gen_program`, `gen_function` e `gen_subroutine`. Depois, para cada nó da AST, identifica o seu tipo, por exemplo `ASSIGN`, `IF`, `DO`, `PRINT`, `CALL` ou operações aritméticas, e encaminha-o para o respetivo gerador de código. Para as potências, chama as funções definidas em código máquina em `utils.py` e depois concatena estas funções ao código gerado.

Durante este processo, o `translator` consulta a tabela de símbolos para obter informação sobre variáveis, tipos, índices e escopos. Também aloca memória para as variáveis e traduz expressões e instruções de Fortran77 para comandos da EWVM, como `PUSHI`, `PUSHG`, `STOREG`, `JUMP`, `CALL`, `WRITEI`, etc.

Assim, o `translator` recebe a estrutura do programa já validada e produz o código máquina correspondente ao original em Fortran77.

7. Testes Realizados

Para além dos testes presentes no enunciado, foram acrescentados mais testes para validar novas implementações. Na tabela abaixo é possível verificar que funcionalidade o ficheiro quer testar.

Teste	Funcionalidade/ Erro testado
ex1.f	PROGRAM, PRINT, string literal
ex2.f	READ, DO, CONTINUE com label, multiplicação
ex3.f	IF/THEN/ENDIF, .AND., .TRUE./ .FALSE., MOD, GOTO, variável LOGICAL
ex4.f	Arrays, leitura para array com DO, soma acumulada
ex5.f	FUNCTION, chamada de função, RETURN, comentários com C, MOD, GOTO
ex6.f	IF/ELSE/ENDIF aninhados, variáveis INTEGER e LOGICAL, comparações
ex7.f	REAL, .NE., variável LOGICAL, operações aritméticas e instruções minúsculas
ex8.f	Comentários (C, *), constantes REAL/científicas, PARAMETER, concatenação, continuação de linha, **, STOP
ex9.f	SUBROUTINE, CALL, passagem de argumentos, RETURN, STOP
ex10.f	Array de inteiros, atribuição de strings, DO com label na mesma linha do corpo, PRINT, STOP

Para além destes testes foram criados mais 25 ficheiros de testes para verificar o lançamento correto de erros. Para estas verificações foram criados ficheiros com código em Fortran 77 com erros propositados, como a utilização de labels não existentes ou tentar dar valores a constantes. Os ficheiros de teste de erros encontram-se no diretório `tests/errors` e o seu nome descreve o erro a testar.

Finalmente, foram criados ficheiros de teste para as otimizações implementadas (presentes no diretório `tests/otimizacoes`), explícitas no capítulo 9. Para estes testes é possível verificar o seu sucesso a partir do código máquina gerado no diretório `output`.

Para cada programa de teste foi executado o compilador, analisada a AST produzida e comparado o comportamento do código EWVM gerado com o resultado esperado. Também foi criado o script `test.sh` que executa todos os testes e verifica se o output é o esperado (se gera uma mensagem de sucesso ou falha para os erros).

8. Dificuldades Encontradas e Limitações Atuais

Durante o desenvolvimento do compilador, surgiram várias dificuldades principalmente devido às diferenças entre a linguagem Fortran77 e o código máquina da EWVM.

Uma parte desafiante foi a implementação das labels. Em Fortran77, as labels são usadas em instruções como GOTO e nos ciclos DO, podendo aparecer no início de linhas e estar associadas ou não a instruções como CONTINUE. Foi necessário garantir que estas labels eram reconhecidas corretamente, guardadas na tabela de símbolos e depois traduzidas para labels válidas em código EWVM.

Outra dificuldade importante foi a implementação de funções e subrotinas. Foi necessário tratar diferentes escopos, parâmetros formais, chamadas com argumentos, valores de retorno e a passagem correta de informação através da stack. No caso das funções, a análise semântica verifica se em algum ponto é atribuído

um valor de retorno à função antes do RETURN. No entanto, esta verificação ainda é limitada, porque não analisa todos os caminhos possíveis de execução.

A implementação de arrays também trouxe dificuldades sobretudo devido à diferença de indexação entre Fortran77 e a EWVM. Em Fortran77, os arrays começam no índice 1, enquanto na EWVM a indexação usada na memória começa em 0. Por isso, sempre que é feito um acesso a um array, foi necessário gerar código extra para converter o índice, subtraindo 1 ao valor usado no programa original.

Também existiram limitações nas instruções de entrada e saída, como o PRINT, READ e WRITE. O PRINT e o READ foram implementados de forma funcional mas ainda com limitações, especialmente no suporte a formatos mais complexos. Já o WRITE não foi implementado, uma vez que a sua sintaxe causava conflitos shift/reduce no parser. Para resolver este conflito, seria preciso realizar alterações significativas na gramática e em várias partes do projeto. Por isso, optou-se por limitar o compilador nesse sentido e focar na implementação de outras funcionalidades e otimizações igualmente importantes.

9. Otimizações Implementadas

Para além de garantir a correta tradução e integridade semântica do código Fortran 77, o compilador inclui otimizações diretamente no **Translator**. O objetivo principal destas transformações é melhorar a eficiência do código gerado para a Máquina Virtual, reduzindo o número de instruções a executar, poupando espaço na memória e eliminando redundâncias computacionais.

9.1. Constant Folding

A constant folding é uma técnica que avalia operações aritméticas ou lógicas entre valores constantes diretamente em tempo de compilação, em vez de gerar código para que a máquina virtual faça esses cálculos em tempo de execução. Durante a tradução do código, são encontradas expressões cujos operandos são literais conhecidos (como números inteiros ou reais), ele calcula o resultado imediatamente e substitui todo o nó da operação por um único nó. Por exemplo:

Para o código

```
X = 3 + 5 * 2
```

Seriam gerados os seguintes comandos:

```
PUSHI 3 -> PUSHI 5 -> PUSHI 2 -> MUL -> ADD -> STOREG 0
```

Com a otimização, o **Translator** faz os cálculos e seria apenas gerado:

```
PUSHI 13 -> STOREG 0
```

9.2. Eliminação de Código Morto

A otimização de código morto é implementada no **Translator** em estruturas condicionais IF e foca-se em limpar o programa de quaisquer blocos de instruções que nunca serão alcançados pela execução.

Se o tradutor determinar que a condição de um IF é estritamente falsa, todo o bloco de instruções principal (THEN) é ignorado e descartado do ficheiro final, gerando apenas código relativo ao ELSE. Por exemplo:

Para o código

```
IF ( .FALSE. ) THEN
    X = 10
ELSE
    X = 20
ENDIF
```

seria gerado:

```
PUSHI 0 -> JZ IF1ELSE -> PUSHI 10 -> STOREG 0 -> JUMP IF1END -> IF1ELSE: -> PUSHI 20 ->
STOREG 0 -> IF1END:
```

é apenas gerado `PUSHI 0 -> JZ IF1ELSE -> IF1ELSE -> PUSHI 20 -> STOREG 0`

9.3. Remoção de Ciclos Mortos

A remoção de ciclos mortos deteta loops DO que nunca chegam a realizar uma única iteração. Ao analisar as instruções de um ciclo, o tradutor compara o limite inicial com o limite final fornecido e tem em consideração a direção do step.

Se for detetado um ciclo onde o valor inicial já ultrapassou o objetivo logo à partida (como no exemplo abaixo), o **Translator** reconhece que é código que nunca será executado e, em vez de gastar memória a criar variáveis de iteração na stack e a gerar etiquetas de ciclo, ignora completamente as instruções do ciclo.

Para o código

```
DO 10 I = 10, 1, 1
    X = X + I
10 CONTINUE
```

não é gerado nenhum comando para este ciclo, já que 10 é maior que 1 e é um ciclo com um incremento positivo.

9.4. Eliminação de Dupla Negação

A eliminação de dupla negação atua especificamente sobre expressões lógicas e booleanas que possuam inversões consecutivas.

Ao intersetar este padrão lógico durante a tradução, o **Translator** anula ambas as operações. Na máquina virtual, isto traduz-se diretamente na remoção de instruções repetidas **NOT**, otimizando o fluxo lógico que decide os desvios no programa. Por exemplo:

Para o código

```
IF (.NOT. (.NOT. FLAG)) THEN
```

Sem otimizações, seria gerado: `PUSHG 2 -> NOT -> NOT -> JZ IF2END`

Com a otimização: `PUSHG 2 -> JZ IF2END`

10. Instruções de Execução

Para executar o compilador, é necessário ter o Python instalado e instalar a biblioteca PLY:

```
pip install ply
```

1. Para executar é necessário estar no diretório `<project_dir>`, onde se encontra o ficheiro `main.py`.
2. Para executar o compilador e gerar o ficheiro de output com o código máquina EWVM, deve ser passado como argumento o ficheiro Fortran77:

```
python3 main.py <ficheiro.f>
```

11. Conclusão

O desenvolvimento deste compilador permitiu concretizar a conceção e implementação de uma ferramenta capaz de traduzir um subconjunto da linguagem Fortran 77 para o código máquina da EWVM. Ao longo do projeto, foram consolidadas de forma prática as várias fases que compõem a arquitetura de um compilador, desde a análise léxica até à geração de código final.

A gramática foi capaz de resolver de forma elegante a precedência de operadores e de lidar com ambiguidades sintáticas intrínsecas ao Fortran 77, como a distinção entre acessos a arrays e chamadas de funções. A conceção da Tabela de Símbolos assumiu um papel central na arquitetura do sistema por garantir uma validação rigorosa de tipos, escopos e inicializações, e também por servir de fundação para a tradução e alocação de memória na máquina virtual.

Ainda foi possível a implementação de otimizações que demonstraram um impacto direto na redução do número de instruções geradas, resultando num código máquina EWVM mais compacto e, consequentemente, com menor gasto de memória e um tempo de execução menor.

Naturalmente, foram encontradas algumas limitações, como a ausência de suporte a `WRITE` e de diretivas `FORMAT`. Estas limitações são decorrentes, sobretudo, da necessidade de reestruturação profunda do código e ao tempo necessário para a implementação correta.

Ainda assim, o trabalho desenvolvido cumpriu os objetivos propostos e permitiu consolidar conhecimentos fundamentais sobre o funcionamento interno de um compilador, bem como aprofundar o conhecimento sobre uma linguagem que era inicialmente desconhecida pelo grupo.